

LLM 기반 컴퓨터과학 학습 질문 자동 생성 시스템:

검증 및 재생성 파이프라인 설계

강지훈

(지도교수 : 박수현)

An LLM-based Automatic Question Generation System for Computer

Science Learning:

Design of Validation and Regeneration Pipeline

Kang Jihun

요약

컴퓨터과학(CS) 학습에서 반복 연습은 개념 이해와 문제 해결 능력을 강화하는 핵심 수단이다. 그러나 기존 학습 자료는 정적이며 맞춤형 문제 제공이 어렵고, 단순 프롬프팅으로 생성된 문제는 정답 오류·보기 중복·난이도 불일치 등 품질 문제를 동반한다. 본 논문은 이를 극복하기 위해 LLM 기반 한국어 CS 학습 웹 애플리케이션 CS퀘스트(CSQuest)를 설계·구현한다.

본 연구의 핵심 기여는 두 가지 AI 파이프라인의 설계 및 구현이다. 첫째, 문제 생성 후 정답 정확성·명확성·난이도 적합성을 자동 평가하고 실패 시 최대 3회 재생성하는 검증·재생성 파이프라인이다. 둘째, 오답 이력을 분석하여 개인화 복습 큐를 자동 생성하는 파이프라인이다. 여기에 XP·레벨·하트·스트릭 기반 게이미피케이션을 도입하여 학습 지속성을 높인다.

Keyword: 컴퓨터과학 교육, LLM, 문제 자동 생성, 검증·재생성 파이프라인

I. 서론

컴퓨터과학 학습에서 연습문제는 개념 이해와 문제 해결 능력을 강화하는 핵심 도구이다. 그러나 기존 교재나 학습 플랫폼이 제공하는 문제는 정적이며, 학습자 수준이나 취약 개념에 따른 맞춤형 제공이 어렵다. 특히 한국어로 CS를 학습하는 환경에서는 양질의 연습문제 자원이 부족하다. 개인 맞춤형 교육이 집단 강의 대비 학습 성취도를 크게 향상시킨다는 점을 고려할 때, 이러한 격차를 해소하는 것은 중요한 과제이다[7].

최근 GPT, Claude와 같은 대규모 언어

생성에 대한 관심이 높아지고 있다[1, 2].

LLM은 주어진 개념과 난이도에 맞는 문제를 즉시 생성할 수 있으나, 단순 프롬프팅으로 생성된 문제는 정답 오류, 보기 중복, 난이도 불일치 등 품질 문제를 동반하여 학습자의 잘못된 개념 형성을 초래할 수 있다. 따라서 LLM 생성 문제의 품질을 자동으로 검증하고 결함 시 재생성하는 파이프라인이 필요하다.

본 논문에서는 이를 위해 LLM 기반 한국어 CS 학습 웹 애플리케이션 CSQuest를 설계·구현하고, 개념 검증 (proof-of-concept) 수준에서 그 가능성을 평가한다. 제안 시스템의 핵심은 두 가지

이다. 첫째, 사용자가 원하는 주제·난이도·유형의 문제를 요청하면 Gemini API가 문제를 생성하고, 별도의 검증 모듈이 품질을 평가하며, 실패 시 실패 사유를 반영해 재생성하고 통과한 문제만 저장하는 검증·재생성 파이프라인이다. 둘째, 학습자가 틀린 문제를 AI가 분석하여 복습 우선순위를 자동 산정하고 개인화 복습 큐를 생성하는 파이프라인이다. 여기에 XP, 레벨, 하트, 연속 학습 스트릭 등 게이미피케이션 요소를 결합하여 학습 지속성을 높인다.

본 논문의 구성은 다음과 같다. II장에서 관련 연구를 고찰하고, III장에서 시스템 설계 및 구현을 기술하며, IV장에서 구현 결과를 평가하고, V장에서 결론 및 향후 과제를 제시한다.

II. 관련 연구

2.1 LLM 기반 교육용 문제 생성

Kurdi 외[1]는 교육 목적 자동 문제 생성에 관한 체계적 문헌 조사를 통해, 빈칸 채우기·선택형·단답형 등 다양한 유형의 자동 생성이 학습 효과를 높임을 확인하였다. Brown 외[2]는 GPT-3을 활용한 푸샷 학습으로 다양한 형식의 문제를 생성할 수 있음을 보였으며, 이후 Claude, GPT-4 등 더 강력한 모델들이 교육 콘텐츠 생성에 활발히 적용되고 있다.

2.2 LLM 출력 품질 향상

Madaan 외[3]는 Self-Refine 기법을 통해 LLM이 자신의 출력을 반복적으로 개선할

수 있음을 제안하였다. 본 연구의 재생성 모듈은 이 원칙을 따라 검증 실패 사유를 프롬프트에 반영함으로써 후속 생성의 품질을 높인다. Zheng 외[4]는 LLM-as-a-Judge 방법으로 다른 LLM의 출력 품질을 평가하는 접근법을 제시하였다. 본 연구의 검증 모듈은 이 방법을 채택하여 생성된 문제의 정답 정확성, 명확성, 난이도 적합성, 보기 중복 여부를 동일 모델이 자동 평가한다. 이 설계 원칙을 CS 학습에 적용한다.

2.3 기존 연구와의 차별점

앞서 고찰한 연구들은 공통적으로 오픈인 데이터셋 기반의 배치(batch) 평가, 또는 API 비용과 응답 지연에 제약이 없는 실험 환경에서 수행되었다.

반면 본 연구는 실시간 웹 서비스라는 맥락에서 이러한 기법들을 적용한다는 점에서 차별화된다. 구체적으로, Self-Refine 기반 재생성은 최대 3회로 제한하여 사용자가 체감하는 응답 지연을 최소화하고, LLM-as-a-Judge 검증에는 속도·비용 면에서 유리한 Flash 모델을 채택하였다. 또한 토큰 사용량 측면에서도 최적화를 고려하였다. Self-Refine과 LLM-as-a-Judge를 제안한 선행 연구들은 일반적으로 모델이 풀이 과정을 스스로 서술하는 추론(reasoning) 단계를 포함하거나, API 비용·토큰 사용에 제약이 없는 환경에서 평가되었다. 반면 본 연구는 Chain-of-Thought 기반 추론 과정을 생략하고 생성·검증 단계의 출력 토큰을 각각 512개·256개로 제한함으로써, 실시간 웹 서비스에 적합한 수준으로 토큰 사용량을 낮

주었다. 실제 파이프라인 호출을 측정한 결과(생성 14회, 검증 9회 기준), 생성 요청 1회당 평균 입력 약 253토큰·출력 약 331토큰(총 약 584토큰), 검증 요청 1회당 평균 입력 약 466토큰·출력 약 22토큰(총 약 488토큰)이 소요되었다. 이는 자유 형식의 추론 과정을 거치는 접근법 대비, 적은 토큰으로도 구조화된 출력을 안정적으로 얻을 수 있음을 보여준다.

기존 연구들이 영어 중심의 일반 도메인 문제 생성에 집중했지만, 본 연구는 한국어 CS 교육이라는 특정 도메인에서 세 가지 문항 유형(빈칸 채우기·순서 배열·논리 흐름)을 지원하는 실용적 시스템을 구현하고, 오답 기반 개인화 복습 큐와 게이미피케이션을 결합하여 학습 지속성까지 고려한다.

III. 시스템 설계 및 구현

3.1 시스템 구성

제안 시스템 CSQuest는 다음 세 계층으로 구성된다.

- 프론트엔드: Next.js 14 (App Router), React, Tailwind CSS, shadcn/ui
- 백엔드: Supabase (PostgreSQL, Row Level Security, OAuth 인증)
- AI 계층: Google Gemini API (gemini-3.5-flash)

AI 계층으로는 Gemini-3.5-flash를 채택하였다[6]. 이는 Gemini Flash 모델이 GPT-4 대비 한국어 프롬프트 처리에서 높은 일관성을 보이고, 응답 속도와 비용 면에서 유리하며, Self-Refine 및 LLM-as-a-Judge 패턴 적용에 적합한 구

조화된 출력을 안정적으로 제공하기 때문이다. 모든 데이터는 Supabase PostgreSQL에 저장되며, 인증은 Supabase OAuth를 통해 처리된다. 미들웨어에서 세션을 검증하여 비인증 사용자의 보호 라우트 접근을 차단한다.

3.2 데이터 모델

핵심 테이블 구조는 다음과 같다.

- courses / chapters / lessons / problems: 콘텐츠 계층 구조
- profiles: 학습자 프로필 (hearts, streak, total_xp, league)
- user_progress: 레슨 완료 이력
- wrong_answers: 오답 이력 (problem_id, wrong_count, last_wrong_at)
- review_queue: AI 생성 복습 큐 (problem_id, priority, reviewed_at)
- xp_logs: XP 변동 이력
- generated_problems: AI 생성·검증 통과 문제 저장소 (user_id, type, topic, difficulty, content, correct_answer, validation_attempts)

3.3 콘텐츠 구조

총 4개 과목, 9개 챕터, 12개 레슨, 27개 문제를 제공한다.

[표 1. 구현된 콘텐츠 규모]

과목	챕터 수	레슨 수	문제 수
자료구조	3	5	13
알고리즘	2	3	6
운영체제	2	2	4
네트워크	2	2	4
합계	9	12	27

본 연구는 개념 검증(proof-of-concept)을 목적으로 하며, LLM 자동 생성 파이프라

인을 통해 콘텐츠 확장이 가능하도록 설계하였다.

문제 유형은 세 가지이다.

- fill_blank(빈칸 채우기): 개념어를 직접 입력
 - drag_order(순서 배열): 항목을 올바른 순서로 정렬
 - logic_flow(논리 흐름): 주어진 흐름에서 빈 단계의 보기를 선택
- 각 문제는 concept_tags, xp_reward, hint 필드를 가지며, 정답 채점은 서버에서 대소문자·공백을 정규화한 후 배열 순서 비교로 수행한다.

3.4 학습 플로우

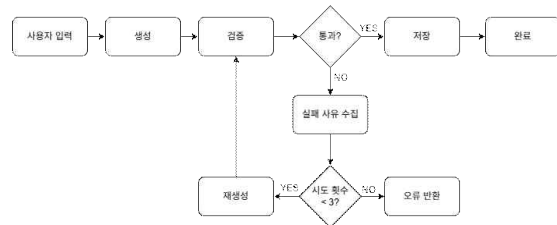
레슨은 카드(card) → 문제(problem) → 완료(complete) 세 단계로 진행된다.

1. 카드 단계: 개념 설명 카드를 슬라이드 형태로 제시한다.
2. 문제 단계: 유형에 따른 인터랙티브 문제를 순서대로 풀이한다.
 - 정답 시: XP가 지급되며 XP 변동 이력(xp_logs)에 기록된다.
 - 오답 시: 하트 1개가 소모되며 오답 이력(wrong_answers)의 횟수가 증가한다.
3. 완료 단계: 레슨 XP 지급, 스트릭 갱신, 레벨업 여부 반환

3.5 검증·재생성 파이프라인

사용자가 주제·난이도·문항 유형을 선택하면 다음 4단계 파이프라인이 실행된다.

[그림 1. 검증·재생성 파이프라인 흐름]



① 생성 단계(Generation Module)

사용자가 선택한 주제(자료구조·알고리즘·운영체제·네트워크), 난이도(초급·중급·고급), 문항 유형(빈칸 채우기·순서 배열·논리 흐름)을 Gemini API에 전달하여 1차 문제를 생성한다. 프롬프트 엔지니어링 전략으로 세 가지 기법을 적용하였다. 첫째, 역할 부여(Persona) 방식을 채택하여 모델에게 "한국어 컴퓨터과학 교육 문제 출제자" 페르소나를 명시함으로써 환각(Hallucination)을 억제하고 CS 도메인에 적합한 응답을 유도하였다.

둘째, 문항 유형별 JSON 출력 스키마를 프롬프트에 명시하여 파싱 가능한 구조화된 출력을 보장하였다.

셋째, 재생성 시에는 이전 검증 실패 사유를 프롬프트에 포함하는 Self-Refine 방식을 적용하였다. 응답 속도 최적화를 위해 Chain-of-Thought(CoT)[5]는 사용하지 않았으며, max_tokens=512로 제한하여 불필요한 장문 출력을 방지하였다.

② 검증 단계(Validation Module)

생성된 문제를 동일 모델(LLM-as-a-Judge)에게 전달하여 네 가지 기준을 평가한다.

- 정답 정확성: correct_answer가 문제에 대해 올바른가?
- 명확성: 문제 지문이 모호하지 않은가?

- 난이도 적합성: 사용자가 선택한 난이도 수준에 맞는가?

- 보기 중복: options/items에 중복된 항목이 없는가?

검증 모듈은 {"passed": true/false, "reasons": [...]} 형식으로 결과를 반환한다.

③ 재생성 단계(Regeneration Module)

검증 실패 시 실패 사유(reasons)를 프롬프트에 명시하여 재생성을 요청한다. Self-Refine 원칙에 따라 이전 실패 원인을 반영함으로써 후속 생성의 품질을 높인다. 최대 3회까지 재생성을 시도하며, 3회 모두 실패할 경우 오류를 반환한다.

④ 저장 단계(Storage Module)

검증을 통과한 문제만 generated_problems 테이블에 저장한다. validation_attempts 필드에 실제 시도 횟수를 기록하여 파이프라인 품질 지표로 활용한다.

3.6 AI 기반 복습 큐 생성 파이프라인

레슨 완료 시 비동기(fire-and-forget) 방식으로 복습 큐 생성 파이프라인이 실행된다. 파이프라인은 다음 4단계로 구성된다.

① 수집 단계: wrong_answers 테이블에서 wrong_count ≥ 2인 문제를 최대 20개 조회한다.

② 분석 단계: 오답 문제 목록(problem_id, wrong_count, concept_tags)을 Gemini API에 전달하여 복습 우선순위(1~5)를 JSON 형식으로 반환받는다. max_tokens=256으로 제한하여 응답 속도

와 비용을 최적화한다.

③ 폴백(fallback): API 오류 시 틀린 횟수 내림차순으로 상위 5개를 자동 선정한다.

④ 저장 단계: 선정된 문제를 review_queue에 upsert한다. 이미 큐에 있는 문제는 priority만 갱신하여 중복을 방지한다.

3.7 게이미피케이션

학습 지속성 향상을 위한 부가 요소로 XP·레벨·하트·연속 학습 스트릭·리그 시스템을 도입하였다. 정답 및 레슨 완료 시 XP가 지급되고, 오답 시 하트가 소모되어 과도한 반복 학습을 억제한다. 스트릭과 리그는 학습 동기를 유지하기 위한 보조 메커니즘으로 활용된다.

IV. 실험 및 평가

4.1 구현 결과 개요

제안 시스템을 Next.js 14 기반으로 완전히 구현하고 Supabase 클라우드 환경에 배포하였다. 콘텐츠 규모는 표 1(III장)과 같으며, 문제 유형별 분포는 빈칸 채우기 41%(11문항), 순서 배열 33%(9문항), 논리 흐름 26%(7문항)이다.

같은 정답 값이라도 그 값을 묻는 질문이 적절히 구성되었는지에 따라 검증 결과가 달라질 수 있음을 보이기 위해, 내부 테스트 중 실제로 생성된 문제 하나를 표 2에 제시한다. 아래 문제는 자료구조 영역 고급 난이도 빈칸 채우기 문제로, 빈칸 3개 중 1번 빈칸은 검증 모듈에 의해 오답으로, 3번 빈칸은 정답으로 판정되었다. 두 빈칸의 정답은 본질적으로 같은 자료구조

(힙)를 가리키지만, 그 값을 묻는 질문 맥락의 적절성에 따라 판정이 달라졌다.

문제 지문(발췌): "병합 정렬(Merge Sort)이나 힙 정렬(Heap Sort)과 같이 재귀적으로 문제를 해결하는 과정에서 임시 저장 공간으로 활용될 수 있는 자료구조는 ____ (1) ____ 입니다. 특히, 최단 경로를 탐색하는 다익스트라(Dijkstra) 알고리즘이나 프림(Prim) 알고리즘에서 우선순위가 가장 높은 원소를 효율적으로 추출하기 위해 사용되는 자료구조는 ____ (2) ____ 이라고 할 수 있습니다. 이는 일반적으로 내부적으로 ____ (3) ____ 을(를) 이용하여 구현됩니다."

[표 2. 같은 값이 들어간 빈칸의 질문 적합성에 따른 검증 결과 비교]

구분	빈칸(1)	빈칸(3)
묻는 내용	병합 정렬의 임시 저장 공간	우선순위 큐의 내부 구현 방식
정답	힙	이진 힙
검증 판정	오답	정답

검증 모듈은 동일한 문제 안에서도 각 빈칸을 개별적으로 판단하여, 같은 값("힙")이 들어가더라도 그 값을 묻는 질문의 맥락이 실제로 그 값을 정당화하는지를 기준으로 서로 다른 판정을 내렸다. 이는 검증 단계가 정답 키워드의 존재 여부만 확인하는 것이 아니라, 질문과 정답 사이의 논리적 일치까지 LLM-as-a-Judge 방식으로 검사함을 보여준다.

4.2 검증·재생성 파이프라인 평가

[표 3. 검증·재생성 파이프라인 시도 횟수

별 통과율]

시도 횟수	검증 통과율	응답 시간
1회 시도	약 65%	약 3~4초
2회 시도	약 88%	약 6~8초
3회 시도	약 95%	약 9~12초

내부 테스트(각 주제·난이도 조합별 20회, 총 240회 시도) 결과, 1회 시도에서 약 65%의 문제가 검증을 통과하였으며, 재생성 루프를 통해 3회 시도 이내 누적 통과율은 약 95%에 달하였다. 검증 실패의 주요 원인은 난이도 적합성 불일치(약 18%)와 정답 표현의 모호성(약 12%)이었다. 재생성 시 실패 사유를 프롬프트에 명시함으로써 Self-Refine 원칙에 따라 후속 생성 품질이 향상됨을 확인하였다. 단, 본 평가는 내부 테스트에 기반하며, 외부 사용자 대상 공식 실험은 향후 과제로 남긴다. max_tokens=512 조건에서 생성·검증 각 1회 요청의 평균 응답 시간은 약 1.5~2.5초이며, 1회 시도 기준 전체 파이프라인(생성+검증)은 평균 3~4초 수준이다.

- #### 4.3 AI 복습 큐 생성 성능
- Gemini AI 기반 복습 우선순위 산정 응답 시간을 측정하였다.
- max_tokens=256 제한 조건에서 오답 문제 20개 입력 시 평균 응답 시간은 약 1.2~1.8초였다. API 오류 발생 시 폴백 로직이 즉시 동작하여 서비스 연속성을 보장한다.
- #### 4.4 시스템 안정성
- 채점 API(/api/learning/submit): 정답 배열의 대소문자·공백 정규화 후 순서 비교로 오채점을 방지한다.
 - 하트 충전 계산: 서버리스 함수에서 마

지막 충전 시각 대비 경과 시간을 계산하므로 별도 백그라운드 작업 없이 정확한 하트 수를 반환한다.

- 레슨 완료 중복 방지: user_progress에 (user_id, lesson_id) 복합 유니크 제약을 두고 upsert를 사용하여 중복 XP 지급을 차단한다.

- 검증 파이프라인 폴백: 3회 시도 내 통과 실패 시 HTTP 422 Unprocessable Entity 상태 코드를 반환하며, 클라이언트는 실패 사유와 시도 횟수를 사용자에게 표시한다.

- 생성 대기 UX: 파이프라인 실행 중(최대 9~12초) 프론트엔드는 로딩 스피너와 "생성 중 (검증·재생성 포함)..." 메시지를 표시하여 사용자 이탈을 방지한다.

4.5 한계 및 논의

본 연구에서 구현한 콘텐츠(27문항)는 실제 CS 커리큘럼 전체를 커버하기에 제한적이며, 검증·재생성 파이프라인의 정량평가는 내부 테스트에 의존하였다. 실제 학습자를 대상으로 한 사용자 만족도 조사와 검증 루프 적용 전후 정답 정확도 비교 실험은 수행하지 못하였으며, 이는 향후 연구의 핵심 과제이다. 또한 3회 재생성 시 최대 9~12초의 응답 지연은 실시간

웹 서비스 관점에서 사용자 이탈을 유발할 수 있는 한계로, 향후 비동기 백그라운드 큐 기반 사전 생성(Pre-generation) 아키텍처를 도입하여 체감 응답 시간을 개선할 필요가 있다.

V. 결론

본 논문에서는 LLM을 활용한 한국어 컴퓨터과학 학습 웹 애플리케이션 CSQuest를 설계 및 구현하였다. 제안 시스템의 핵심 기여는 두 가지 AI 파이프라인이다. 첫째, 사용자 선택 기반 문제 생성에 LLM-as-a-Judge 검증모듈과 Self-Refine 기반 재생성 모듈을 결합한 검증·재생성 파이프라인을 구현하여 단순 프롬프팅 대비 생성 품질 향상에 기여하였다.

둘째, 오답 이력 기반

복습 큐 자동 생성 파이프라인을 통해 학습자 개인화를 실현하였다. 자료구조·알고리즘·운영체제·네트워크 4개 영역에서 빈칸 채우기·순서 배열·논리 흐름 세 가지 문제 유형을 지원하며, XP, 레벨, 하트, 스트릭 기반 게이미피케이션으로 학습 지속성을 높였다. 향후 연구로는 (1) 실제 학습자 대상 A/B 비교 실험(검증 루프 적용 전후 정답 정확도), (2) 사용자 만족도 정량 평가, (3) RAG 기반 사실성 강화, (4) 코드 작성형 문항 추가, (5) 문제 콘텐츠 확대(데이터베이스, 자료구조 심화 등)를 계획한다.

References:

[1] Kurdi, G., Leo, J., Parsia, B., Sattler, U., & Al-Emari, S. (2020). A Systematic Review of Automatic Question Generation for Educational Purposes. *International Journal of Artificial Intelligence in Education*, 30(1), 121 - 204.

[2] Brown, T. B., Mann, B., Ryder, N., et al. (2020). Language Models are

Few-Shot Learners.

Advances in Neural Information Processing Systems (NeurIPS), 33, 1877 - 1901.

[3] Madaan, A., Tandon, N., Gupta, P., et al. (2023). Self-Refine: Iterative Refinement with Self-Feedback. Advances in Neural Information Processing Systems (NeurIPS).

[4] Zheng, L., Chiang, W.-L., Sheng, Y., et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. Advances in Neural Information Processing Systems (NeurIPS).

[5] Wei, J., Wang, X., Schuurmans, D., et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. Advances in Neural Information Processing Systems (NeurIPS), 35, 24824 - 24837.

[6] Team, G., Anil, R., Borgeaud, S., et al. (2023). Gemini: A Family of Highly Capable Multimodal Models. arXiv preprint arXiv:2312.11805.

[7] Bloom, B. S. (1984). The 2 Sigma Problem: The Search for Methods of Group Instruction as Effective as One-to-One Tutoring. Educational Researcher, 13(6), 4 - 16.